

Code Audit Report

Repository: /Users/mac/Downloads/zynxis-internship-faozan

Audited: 14 August 2026, 23:27

Scope: 37 files · 5,583 lines

Findings: CRITICAL: 3 · HIGH: 5 · MEDIUM: 8 · LOW: 18

Sources: llm-review: 30 · static-analysis: 4

LLM usage: 0 calls · 7 cache hits · 0 retries · 0 tokens

Findings carry a confidence level. `confirmed` means the analyser proved it from the syntax tree or an exact credential pattern. `advisory` means a language model suggested it and could not run the code. Treat them differently.

Executive Summary

The codebase is in decent shape, with 5583 lines of code scanned and a total of 34 findings, of which only 3 are critical. The most serious issue is the use of `eval()` in `Week\_1/react\_agent.py`, which poses a significant security risk. The problems identified do not appear to be systemic, but rather isolated issues that can be addressed through targeted fixes. However, the presence of multiple high-severity findings, including potential code injection vulnerabilities and uncaught exceptions, suggests that the codebase could benefit from additional review and testing. Overall, the codebase requires attention to address these issues, but it is not in a state of crisis.

Themes

The findings suggest two main themes: error handling and input validation. Many of the issues identified, including uncaught exceptions, missing input validation, and potential code injection vulnerabilities, can be traced back to inadequate error handling and input validation. Additionally, there is a theme of resource management, with several findings related to potential resource leaks. These themes are not universal, but they are common enough to suggest that the codebase could benefit from a review of its error handling and input validation practices.

Recommended Priorities

1. **Address critical issues:** Fix the use of `eval()` in `Week\_1/react\_agent.py` and address the uncaught exceptions in `Week\_7/ops\_agent.py` and `Week\_8/auditor/static.py`, as these pose the most significant risks to the codebase. 2. **Review error handling in `ops\_agent.py` and `ingest.py`:** These files have multiple findings related to uncaught exceptions and error handling, suggesting that a thorough review of their error handling practices is necessary. 3. **Implement input validation in `react\_agent.py` and `crew.py`:** These files have findings related to missing input validation, which could lead to security vulnerabilities or data corruption. 4. **Review resource management in `auditor/llm.py` and `ops\_agent.py`:** These files have findings related to potential resource leaks, which could lead to performance issues or crashes. 5. **Conduct a thorough code review of `Week\_8/auditor/static.py`:** This file has multiple low-severity findings, suggesting that it may benefit from a thorough review to identify and address any potential issues.

Findings by Severity

Severity	Count	What it means
CRITICAL	3	Exploitable now, or a live credential. Fix before merging.
HIGH	5	A serious defect or a dangerous pattern. Fix this sprint.
MEDIUM	8	A real problem with a workaround or narrower blast radius.
LOW	18	Worth tidying; unlikely to cause an incident alone.

Findings

CRITICAL

PY-EVAL — Use of eval()

`Week\_1/react\_agent.py:29` · static-analysis · confidence: confirmed

eval() executes arbitrary code. If any part of its argument can be influenced by user input, this is remote code execution.

```
result = eval(expression, {"__builtins__": {}}, allowed)
```

Fix: Replace eval() with explicit parsing — ast.literal_eval for data, or a dispatch dict for behaviour.

LLM-REVIEW — Uncaught Exception in File Operations

`Week\_7/ops\_agent.py:123` · llm-review · confidence: advisory

The code does not handle exceptions that may occur when writing to the output file. If an error occurs while writing to the file, the program will crash without providing any useful information.

Fix: Add a try-except block around the file writing operations to catch and handle any exceptions that may occur.

LLM-REVIEW — Potential SyntaxError Handling

`Week\_8/auditor/static.py` · llm-review · confidence: advisory

The analyse_source function catches a SyntaxError when parsing the source code, but it does not handle other potential exceptions that may occur during the parsing process, such as MemoryError or RecursionError.

Fix: Add additional exception handling to the analyse_source function to handle other potential exceptions.

HIGH

LLM-REVIEW — Potential Code Injection Vulnerability

`Week\_1/react\_agent.py:24` · llm-review · confidence: advisory

The calculator function uses eval() to evaluate mathematical expressions. This can lead to code injection vulnerabilities if the input is not properly sanitized. An attacker could potentially inject malicious code by crafting a specific input string.

Fix: Use a safer evaluation method, such as using a parsing library or a restricted evaluation environment, to prevent code injection attacks.

LLM-REVIEW — Uncaught Exception in Collection Deletion

`Week\_4/ingest.py:63` · llm-review · confidence: advisory

The code catches all exceptions when deleting a collection, but does not re-raise or log them. This means that if an error occurs during deletion, it will be silently ignored, potentially leaving the system in an inconsistent state.

Fix: Log or re-raise the exception instead of silently ignoring it, e.g., `except Exception as e: print(f'Error deleting collection: {e}')`

LLM-REVIEW — Unvalidated Input

`Week\_5/crew.py:145` · llm-review · confidence: advisory

The function phase_brief does not validate the input topic. If topic is empty or None, it may cause unexpected behavior or errors in the subsequent tasks.

Fix: Add input validation to check if topic is not empty and not None before proceeding with the task.

LLM-REVIEW — Potential Data Loss Due to Unhandled Errors

`Week\_7/ops\_agent.py:105` · llm-review · confidence: advisory

The code catches stages.StageError and sched.ScheduleError exceptions, but it does not provide any information about the error. If an error occurs, the program will exit without providing any useful information, potentially leading to data loss.

Fix: Modify the exception handling code to log or display the error message, and consider adding a mechanism to save any unsaved data before exiting the program.

LLM-REVIEW — Potential Resource Leak

`Week\_8/auditor/llm.py:123` · llm-review · confidence: advisory

The file descriptor for the cache file is not explicitly closed after writing. If an exception occurs during the write operation, the file descriptor may remain open, leading to a resource leak.

Fix: Use a ``with`` statement to ensure the file descriptor is properly closed after writing, e.g., ``with self._cache_path(payload).open('w') as f: f.write(json.dumps({'response': response}))``

MEDIUM

LLM-REVIEW — Missing Input Validation

``Week_1/react_agent.py:43`` · llm-review · confidence: advisory

The lookup function does not validate its input. If the input query is None or empty, the function may throw an exception or return unexpected results.

Fix: Add input validation to check for None or empty input and handle these cases accordingly.

LLM-REVIEW — Potential KeyError

``Week_1/react_agent.py:64`` · llm-review · confidence: advisory

The TOOLS dictionary is accessed without checking if the tool_name exists. If the tool_name is not in the TOOLS dictionary, a KeyError will be thrown.

Fix: Add a check to ensure the tool_name exists in the TOOLS dictionary before accessing it.

LLM-REVIEW — Potential Resource Leak

``Week_4/ingest.py:74`` · llm-review · confidence: advisory

The chromadb client is not explicitly closed, which could lead to resource leaks if the program is terminated abruptly.

Fix: Add a ``finally`` block to close the client, e.g., ``try: ... finally: client.close()``

LLM-REVIEW — Potential KeyError

``Week_5/crew.py:193`` · llm-review · confidence: advisory

In the ``split_report`` function, the code assumes that the section titles in the report match the titles in the ``SECTIONS`` list. If there is a mismatch, a KeyError may occur.

Fix: Add error handling to handle the case where a section title is not found in the ``SECTIONS`` list.

LLM-REVIEW — Potential Issue with Date Parsing

``Week_7/ops_agent.py:73`` · llm-review · confidence: advisory

The code uses `datetime.strptime` to parse the start date from the command line argument. If the date is not in the correct format, a `ValueError` will be raised. However, the code does not handle this exception, which could lead to unexpected behavior.

Fix: Add a try-except block around the date parsing code to catch and handle any `ValueErrors` that may occur.

LLM-REVIEW — Uncaught Exception in `usage` Method

``Week_8/auditor/llm.py:146`` · llm-review · confidence: advisory

The ``usage`` method does not handle potential exceptions that may occur when accessing the ``usage`` attribute of the ``response`` object.

Fix: Add try-except blocks to handle potential exceptions, e.g., ``try: self.prompt_tokens += getattr(usage, 'prompt_tokens', 0) or 0; except Exception as e: print(f'Error accessing usage attribute: {e}')``

LLM-REVIEW — Potential Resource Leak

``Week_8/auditor/static.py`` · llm-review · confidence: advisory

The ``Analyzer`` class does not explicitly close any resources it may open, such as file handles, which could lead to resource leaks.

Fix: Add a ``__del__`` method to the ``Analyzer`` class to ensure that any resources it opens are properly closed.

LLM-REVIEW — Potential Unhandled Edge Case

``Week_8/auditor/static.py`` · llm-review · confidence: advisory

The ``visit_Call`` method in the ``Analyzer`` class does not handle the case where the ``node.func`` attribute is not an instance of ``ast.Name`` or ``ast.Attribute``, which could lead to an `AttributeError`.

Fix: Add additional checks to the ``visit_Call`` method to handle other potential types of ``node.func``.

LOW

LLM-REVIEW — Missing Error Handling

`Week\_1/react\_agent.py:14` · llm-review · confidence: advisory

The `load_dotenv` function is called without error handling. If the `.env` file is missing or cannot be loaded, the program may throw an exception.

Fix: Add try-except blocks to handle potential errors when loading the `.env` file.

LLM-REVIEW — Potential Infinite Loop

`Week\_1/react\_agent.py:83` · llm-review · confidence: advisory

The `run_react` function has a `max_steps` parameter to prevent infinite loops. However, if the `max_steps` parameter is set too high or if the loop condition is not met, the function may still run indefinitely.

Fix: Add additional checks to ensure the loop condition is met and the function terminates as expected.

LLM-REVIEW — Potential Index Out of Range Error

`Week\_4/ingest.py:45` · llm-review · confidence: advisory

The code uses ``range(len(words))`` to iterate over the words in a page, but does not check if the list is empty before accessing its elements.

Fix: Add a check to ensure that the list is not empty before accessing its elements, e.g., ``if words: ...``

LLM-REVIEW — Potential Division by Zero Error

`Week\_4/ingest.py:51` · llm-review · confidence: advisory

The code calculates the step size as ``chunk_words - overlap``, but does not check if the result is zero.

Fix: Add a check to ensure that the step size is not zero, e.g., ``if step == 0: ...``

PY-SWALLOWED-EXCEPT — Exception caught and discarded

`Week\_4/ingest.py:77` · static-analysis · confidence: confirmed

The handler body is just ``pass``, so the failure leaves no trace.

```
except Exception:
```

Fix: Log the exception, even if recovery is genuinely a no-op.

LLM-REVIEW — Inconsistent Error Handling

`Week\_4/ingest.py:81` · llm-review · confidence: advisory

The code checks if ``--reset`` is in ``sys.argv``, but does not handle the case where the argument is not provided. This could lead to inconsistent behavior.

Fix: Use a more robust argument parsing mechanism, such as the ``argparse`` library, to handle command-line arguments.

LLM-REVIEW — Potential Environment Variable Issue

`Week\_5/crew.py:55` · llm-review · confidence: advisory

The code assumes that the ``GROQ_API_KEY`` environment variable is set. If it is not set, the program will exit with an error message. It may be better to handle this case more robustly.

Fix: Add error handling to handle the case where the ``GROQ_API_KEY`` environment variable is not set.

LLM-REVIEW — Potential Resource Leak

`Week\_5/crew.py:105` · llm-review · confidence: advisory

The ``build_llm`` function creates an LLM object, but it is not clear if it is properly closed or released when it is no longer needed. This may cause a resource leak if the function is called multiple times.

Fix: Add a try-finally block to ensure that the LLM object is properly closed or released when it is no longer needed.

LLM-REVIEW — Potential Error Handling Issue

`Week\_5/crew.py:231` · llm-review · confidence: advisory

The ``assemble`` function does not appear to handle errors that may occur during the assembly of the report. If an error occurs, it may not be properly handled or logged.

Fix: Add try-except blocks to handle potential errors that may occur during the assembly of the report.

LLM-REVIEW — Potential Resource Leak

`Week\_7/ops\_agent.py:27` · llm-review · confidence: advisory

The code opens a file for reading, but it does not explicitly close the file. Although the file will be closed when the program exits, it is still good practice to close the file explicitly to avoid potential resource leaks.

Fix: Consider adding a finally block to the try-except block to ensure that the file is closed, regardless of whether an exception is thrown.

LLM-REVIEW — Potential Issue with Team Parsing

`Week\_7/ops\_agent.py:55` · llm-review · confidence: advisory

The code uses the `parse_team` function to parse the team specification from the command line argument. If the team specification is not in the correct format, a `ValueError` will be raised. However, the code does not provide any feedback to the user about the correct format.

Fix: Consider adding a message to the `ValueError` exception to provide feedback to the user about the correct format for the team specification.

PY-SWALLOWED-EXCEPT — Exception caught and discarded

`Week\_8/auditor/llm.py:85` · static-analysis · confidence: confirmed

The handler body is just `pass`, so the failure leaves no trace.

```
except OSError:
```

Fix: Log the exception, even if recovery is genuinely a no-op.

LLM-REVIEW — Potential AttributeError

`Week\_8/auditor/llm.py:173` · llm-review · confidence: advisory

The `__short` function assumes that the `exc` object has a `__str__` method. If `exc` is `None` or does not have a `__str__` method, an `AttributeError` may occur.

Fix: Add a check to ensure `exc` is not `None` and has a `__str__` method before attempting to convert it to a string, e.g., `if exc is not None and hasattr(exc, '__str__'): text = str(exc).replace("\n", ' ')`

PY-SWALLOWED-EXCEPT — Exception caught and discarded

`Week\_8/auditor/llm.py:191` · static-analysis · confidence: confirmed

The handler body is just `pass`, so the failure leaves no trace.

```
except (TypeError, ValueError):
```

Fix: Log the exception, even if recovery is genuinely a no-op.

LLM-REVIEW — Potential TypeError

`Week\_8/auditor/llm.py:191` · llm-review · confidence: advisory

The `__backoff` function assumes that the `retry_after` value is a string that can be converted to a float. If `retry_after` is not a string or cannot be converted to a float, a `TypeError` may occur.

Fix: Add a try-except block to handle potential `TypeError`s, e.g., `try: return min(float(hinted), MAX_DELAY); except (TypeError, ValueError): pass`

LLM-REVIEW — Potential AttributeError

`Week\_8/auditor/llm.py:205` · llm-review · confidence: advisory

The `__is_retryable` function assumes that the `exc` object has a `response` attribute. If `exc` does not have a `response` attribute, an `AttributeError` may occur.

Fix: Add a check to ensure `exc` has a `response` attribute before attempting to access it, e.g., `if hasattr(exc, 'response'): text = str(exc.response).lower()`

LLM-REVIEW — Potential Unhandled Edge Case

`Week\_8/auditor/static.py` · llm-review · confidence: advisory

The `visit_Assign` method in the `Analyzer` class does not handle the case where the `node.value` attribute is not an instance of `ast.Constant`, which could lead to an `AttributeError`.

Fix: Add additional checks to the `visit\_Assign` method to handle other potential types of `node.value`.

LLM-REVIEW — Potential Unhandled Edge Case

`Week\_8/auditor/static.py` · llm-review · confidence: advisory

The `visit\_ExceptHandler` method in the `Analyzer` class does not handle the case where the `node.type` attribute is not an instance of `ast.Name` or `ast.Tuple`, which could lead to an `AttributeError`.

Fix: Add additional checks to the `visit\_ExceptHandler` method to handle other potential types of `node.type`.

Rules Triggered

Rule	Occurrences	Confidence
LLM-REVIEW	30	advisory
PY-EVAL	1	confirmed
PY-SWALLOWED-EXCEPT	3	confirmed

Confidence Levels

Level	Meaning
confirmed	Proven by the syntax tree or an exact pattern match.
probable	Strong signal, but worth a human glance before acting.
advisory	The model's opinion; it could not execute the code.

Suppressed Findings

7 secret-scan finding(s) inside test files were suppressed. Test suites carry fake credentials as fixtures by design; they are listed here so the suppression is visible rather than silent.

- `Week_8/tests/test_secrets.py:35` — Hardcoded credential in source (test fixture)
- `Week_8/tests/test_secrets.py:16` — AWS access key id committed to source (test fixture)
- `Week_8/tests/test_secrets.py:20` — Private key block committed to source (test fixture)
- `Week_8/tests/test_secrets.py:24` — Connection string with password committed to source (test fixture)
- `Week_8/tests/test_secrets.py:35` — AWS access key id committed to source (test fixture)
- `Week_8/tests/test_secrets.py:35` — Credential-shaped assignment to 'secret' (test fixture)
- `Week_8/tests/test_static.py:34` — Credential-shaped assignment to 'api_key' (test fixture)